



Cognizant Programme

Learning Handbook – DE – Java FSE AWS



Programme at a glance

This technical training programme runs for a period of 6 weeks. Please go through the recommended content on the QA Platform and practice the examples, you will also need to complete the hands on and assessment activities on the platform. **At the end of this training, participants should have the following skillset and proficiencies:-**

Stages	Courses/Modules	Key Performance Outcomes
Stage 1	Core Java	After Completing this module, the learner will be able to: • Write modern, clean Java code • Work proficiently with collections, generics & streams • Handle errors and logging in a production-ready manner • Build robust, concurrent applications • Perform file operations and serialization tasks • Package, build and test Java applications • Understand JVM behavior and performance basics
	JUnit & Test Automation	After Completing this module, the learner will be able to: • Write clean, maintainable unit tests using JUnit 5 • Develop data driven tests using parameterized testing • Mock dependencies using Mockito • Test REST APIs and integration layers • Measure and improve code quality through test coverage • Build production-grade testing strategies
	REST API Using JSON	After Completing this module, the learner will be able to: • Design high quality REST APIs using JSON • Build JSON-based API endpoints • Implement secure REST APIs • Document APIs for consumers • Test APIs thoroughly • Apply operational quality attributes
Stage 2	Amazon Aurora PostgreSQL	After Completing this module, the learner will be able to: • Design optimized relational schemas for real applications • Operate and manage Amazon Aurora PostgreSQL clusters • Build reliable application integrations • Ensure high availability, reliability & security • Monitor and optimize DB workloads • Manage lifecycle & migrations
	JavaScript Essentials	After Completing this module, the learner will be able to: • Write modern, clean JavaScript using ES2020+ features • Interact with the DOM & browser environment effectively • Consume REST APIs and handle JSON payloads • Structure and organize JavaScript codebases • Debug JavaScript applications efficiently • Write basic automated tests for JS utilities
	AWS Core (IAM, VPC, S3, CloudWatch)	After Completing this module, the learner will be able to: • Design secure, scalable cloud foundations using IAM & VPC • Work confidently with S3 for storage and application integration • Monitor AWS workloads effectively • Implement operational & security best practices • Integrate AWS services into application workflows
	AWS Lambda & Serverless	After Completing this module, the learner will be able to: • Build Java-based AWS Lambda functions • Integrate Lambda with key AWS event sources • Secure Lambda workloads • Optimize Lambda performance & reliability • Deploy and manage serverless infrastructure • Apply serverless patterns to real-world problems
	CI/CD with Jenkins	After Completing this module, the learner will be able to: • Build modern CI/CD pipelines using Jenkins • Integrate automated testing & quality analysis • Implement SCM-driven workflows • Package & deliver software artifacts • Deploy applications using CI/CD principles • Secure and maintain Jenkins environments • Troubleshoot & operate pipelines effectively
	Containers & Kubernetes	After Completing this module, the learner will be able to: • Containerize applications using industry best practices • Deploy and manage workloads on Kubernetes clusters • Implement configuration, secrets & persistence • Operate Kubernetes effectively • Secure Kubernetes environments • Use deployment automation tools
	Capstone Project (Integration & Delivery)	After Completing this module, the learner will be able to: • Deliver an end-to-end service: Java API + DB + CI/CD + container/K8s + AWS integration • Meet NFRs (availability, performance, security) • Complete demo & documentation; • Pass code review & operational runbook checks.

Continuous Learning and Continuous Evaluation

Self-Learning

3



Cognizant has collaborated with QA to create learning pathways for your technical training. You will be granted access to the platform and your learning pathway which has self-paced materials from a range of sources. You are expected to complete approximately 4 hours of self-paced learning a day which will be complimented by your 2 X 2hrs of Subject Matter Expert (SME) daily connects. ***You are not expected to complete every minute of all of the self-paced materials*** they are there as a comprehensive list of resources that you can draw from to make sure that you acquire the skills and knowledge as detailed in the subsequent sections of the handbook. ***You are expected to complete all hands on, assessments and Labs that are assigned to you in your learning pathway.***

Continuous Evaluation

Regular assessments and hands-on exercises are generally auto evaluated and in some instances evaluated by SMEs or BU Mentors to track progress, provide feedback, and ensure trainees meet learning objectives. Your SME can answer questions and queries you have of these tasks but should not write code or provide the direct answer. The SME will offer guidance on practical exercises to reinforce learning and ensure effective application of concepts.

Learning Recommendation

- **Engagement:** Candidates are encouraged to actively engage with the course material, taking notes, and asking questions where needed. It is the learners responsibility to acquire the skills and knowledge detailed through the course of the technical training.
- **Practice:** Practice is key to mastering concepts. Candidates should complete the exercises, labs, knowledge checks and projects provided in the courses and by the SME to reinforce their learning.
- **Additional Resources:** Candidates should explore additional resources, such as online tutorials, documentation, and forums, to deepen their understanding and address any areas of confusion.

Module/Week 1: Core Java

Courses/ Modules	Topics	Sub-Topics
Core Java	Java Platform & Language Fundamentals	JDK structure (compiler, runtime, tools); Java module system (JPMS basics); Primitive vs reference types; Memory model fundamentals (stack, heap); Variables, operators, control flow; String APIs & immutability; Modern JVM execution model
	Object-Oriented Programming in Modern Java	Classes, objects, constructors; Encapsulation, inheritance, polymorphism; Abstract classes & interfaces; Method overloading & overriding; Access modifiers; Composition vs inheritance; Equality & hashing (equals(), hashCode())
	Collections, Generics & Functional Programming	Core collections: List, Set, Map; Immutable collections; Generics: type inference, bounded types, wildcards; Lambda expressions & functional interfaces; Stream API: filtering, mapping, reducing; Parallel streams (when to use / avoid); Optional for null-safety; Comparator & method references
	Exception Handling & Logging	Checked vs unchecked exceptions; Custom exceptions; try/catch/finally, try-with-resources; Stack traces & exception chaining; Logging frameworks: SLF4J, Logback basics; Logging patterns (structured logs, correlation IDs)
	Java I/O, NIO.2 & Serialization	Files & Paths API; Streams (byte streams, character streams); Buffered I/O; NIO.2 (non-blocking I/O); Directory operations; Serialization basics; JSON serialization (Jackson overview)
	Concurrency, Multithreading & Asynchronous Programming	Threads & thread lifecycle; Executors & thread pools; Callable, Runnable; Future & CompletableFuture; Synchronization: locks, mutex, volatile; Atomic classes Virtual Threads (Project Loom, if available); Race conditions & deadlocks; Best practices for thread-safe code
	Java Memory & Performance Concepts	Garbage collection (G1GC, ZGC basics); JVM tuning high-level view; Profiling basics (CPU, memory); Measuring performance (JMH overview); Object creation & memory footprint considerations
	Build Tools & Code Quality	Project structure conventions; Maven: dependencies, plugins, packaging; Gradle basics; Unit testing with JUnit 5 (brief intro); Linting & static analysis (Checkstyle, PMD); Clean code principles for Java; JavaDoc documentation basics

	Java Language Features (18-25)	Pattern Matching for switch, Record Patterns, Unnamed Variables and Patterns, Flexible Constructor Bodies, Primitive Types in Patterns; Virtual Threads, Structured Concurrency, Scoped Values; Foreign Function & Memory API, Vector API; Ahead-of-Time Class Loading & Linking, Class Data Sharing (CDS) Enhancements; Compact Object Headers; Stream Gatherers, Custom Intermediate Operations; Sequenced Collections, Sequenced Sets, Sequenced Maps; Implicitly Declared Classes, Instance Main Methods; Key Derivation Function (KDF) API, Quantum-Resistant Algorithms; Generational ZGC, Generational Shenandoah, UTF-8 by Default
--	--------------------------------	--

Module/week 2 (Day 6 to 10) – Rest API using JSON

Courses/Modules	Topics	Sub-Topics
REST API Using JSON	REST Principles & API Design	REST constraints (statelessness, uniform interface, resource-based modeling); Resource vs. representation; URI design best practices; HTTP verbs (GET, POST, PUT, PATCH, DELETE); HTTP status codes & standards; API versioning strategies; Pagination, filtering & sorting patterns; Designing JSON-based request/response models
	JSON Processing Fundamentals	JSON syntax & validation; JSON parsing & serialization mechanisms; Data mapping strategies (DTOs, POJOs); Handling nested JSON, arrays & complex structures; JSON schema validation; Error response JSON structures (RFC-7807 Problem Details format)
	Building REST Endpoints	Controller-level endpoint creation; Request mapping and payload handling; Input validation (constraints, custom validation logic); Response construction (success, error, partial responses); Exception handling patterns for JSON APIs; Idempotency practices; Content negotiation (JSON as primary format)
	API Security & Governance	JWT-based authentication & authorization; OAuth2 / token validation flow; Securing endpoints by role & scope; CORS configuration; Input sanitization & common API security pitfalls (OWASP API Top 10)
	Documentation & Testing	OpenAPI/Swagger documentation; Contract-first vs code-first documentation; Unit testing JSON endpoints; Mocking API dependencies; API testing tools (Postman/Newman, cURL)
	Performance & Operational Considerations	Caching strategies for REST APIs; Rate limiting & throttling; Logging & tracing JSON requests/responses; Error correlation IDs; Metrics: request count, latency, error rate

Module/Week 3 (Day 11 to 15) – Junit and Test Automation

Courses/Modules	Topics	Sub-Topics
JUnit & Test Automation	Fundamentals of Unit Testing	Purpose of unit testing in modern engineering; Characteristics of a “good” unit test (fast, isolated, deterministic); Test Pyramid (Unit → Integration → E2E); Red-Green-Refactor (TDD cycle); Project structure for tests (src/test/java); Naming conventions for test classes & methods
	JUnit 5 Architecture & Essentials	JUnit Platform, JUnit Jupiter, JUnit Vintage; Writing test classes & basic test methods; Important annotations: @Test, @BeforeEach, @AfterEach @BeforeAll, @AfterAll @Disabled Assertions: assertEquals, assertTrue, assertThrows, assertAll Assumptions (assumeTrue, assumeFalse) Repeated tests (@RepeatedTest) Tagging tests (@Tag) for grouping & filtering
	Parameterized Tests	@ParameterizedTest mechanics Using sources: @ValueSource, @EnumSource, @CsvSource, @CsvFileSource Custom argument providers; When to use parameterized tests (data-driven scenarios)
	Test Doubles & Mocking (Mockito)	Test doubles: dummy, stub, spy, mock; Mockito fundamentals: @Mock, @InjectMocks, @Spy Stubbing (when(...).thenReturn(...)) Verifying interactions (verify()) Argument matchers (any(), eq()); Mocking exceptions & edge-case flows; Avoiding over-mocking & anti-patterns
	Testing REST APIs & Integration Layers	Testing service classes & business logic; Mocking repositories & external clients; JSON assertion techniques Test slices (if using Spring Boot): @WebMvcTest, @DataJpaTest Embedded servers vs mocked MVC (conceptual only); Validating error responses & edge cases
	Test Coverage & Quality Gates	Code coverage: line, branch & mutation coverage; Using JaCoCo for coverage reporting; Minimum threshold enforcement; Integrating coverage with CI/CD pipelines; Understanding why 100% coverage ≠ quality
	Test Organization, Maintainability & Best Practices	AAA pattern (Arrange-Act-Assert); Keeping mocks minimal and focused; Using factory methods & builders for test data; Test class naming patterns (*Test, *Should, etc.); Selective test execution (tags, filters, build profiles)

		Avoiding flaky tests; Using temporary folders, test fixtures
--	--	--

Module/Week 4 (Day 16 to 20) – Amazon Aurora PostgreSQL & Javascript

Courses/Modules	Topics	Sub-Topics
Amazon Aurora PostgreSQL	Relational Database Fundamentals	PostgreSQL architecture overview; RDBMS concepts (tables, indexes, constraints); ACID properties; ANSI SQL (DDL, DML, TCL); Primary/foreign keys, relationships; Referential integrity
	Amazon Aurora PostgreSQL Essentials	What is Aurora? (architecture, cluster design, storage engine); Reader/Writer endpoints; Aurora vs RDS PostgreSQL – performance and use-case differences; Aurora Serverless v2 – autoscaling concepts; Parameter groups & cluster parameter groups; DB subnet groups, security groups & networking requirements
	Schema Design & SQL Optimization	Designing normalized schemas (1NF–3NF); Index strategies (B-Tree, Hash, Expression Indexing); Query planning & EXPLAIN ANALYZE; Optimizing joins, WHERE clauses, sorting; Understanding VACUUM, ANALYZE, autovacuum; Performance tuning parameters (work_mem, shared_buffers)
	Data Access & Application Integration	Connecting applications via JDBC; Connection pooling concepts (HikariCP); ORM integration (JPA/Hibernate mapping); Transaction isolation levels; Handling deadlocks & long-running transactions; Prepared statements & protection from SQL injection
	Aurora Serverless, Scaling & High Availability	Auto-scaling replicas; Multi-AZ high availability; Failover and automatic recovery; Reader load balancing; Storage auto-scaling behavior; Backtracking vs backups; Replication concepts
	Security, Monitoring & Maintainability	IAM authentication for PostgreSQL; Secrets Manager for credential rotation; At-rest and in-transit encryption (KMS); CloudWatch metrics; Enhanced monitoring & Performance Insights; Slow query logging; Backup/restore strategies; Disaster recovery patterns (cross-region snapshot replication)
	Migration & Data Lifecycle	Migration options (DMS, pg_dump, pg_restore); Data import/export formats (CSV, JSON, COPY command); Schema migration tools (Flyway/Liquibase); Automating DB versioning; Data lifecycle management & partitioning basics
JavaScript	JavaScript Fundamentals & Runtime	JavaScript engines (V8, SpiderMonkey); Execution model: call stack, heap, event loop; Variables (let, const), block scoping; Primitive vs reference types; Operators, expressions, control flow; Truthy/falsy values; Template literals; Strict mode ("use strict")
	Functions & Modern Syntax	Function declarations vs expressions; Arrow functions & lexical this; Default parameters; Rest & spread operators (...); Destructuring objects & arrays; Higher-order functions (map, filter, reduce); Callbacks vs Promises vs async/await
	Working with the Browser & DOM	Document Object Model basics; Selecting elements (querySelector, getElementById); Manipulating content

		& attributes; Event handling (click, input, submit); Event propagation (capturing/bubbling); Basic form validation; Rendering lists dynamically
	Working with JSON & Calling REST APIs	JSON parsing & serialization (JSON.parse, JSON.stringify); Fetch API for async HTTP calls; GET/POST/PUT/DELETE usage; Handling promises, async/await patterns; Error handling (try/catch, network errors); Working with API response codes; Consuming APIs with dynamic rendering; Cross-Origin Resource Sharing (CORS) - conceptual
	Modules, Packaging & Tooling	JavaScript modules (import / export); ES module vs CommonJS (high-level); NPM basics (installing, using packages); Using bundlers (Vite/Webpack - conceptual); Project structure & environment organization; Linting basics (ESLint)
	Functional & Object-Oriented Patterns	Objects & prototypes; Classes & inheritance; Encapsulation & methods; Pure functions & immutability; Closures & lexical scoping; Avoiding common JS pitfalls
	Error Handling & Debugging	try/catch/finally; Error types (TypeError, ReferenceError, etc.); Browser dev tools: Console Breakpoints Watch expressions Network tab Logging best practices Debugging async code

Module/Week 5 (Day 21 to 25) – AWS Core, Lamda and Serverless

Courses/Modules	Topics	Sub-Topics
AWS Core (IAM, VPC, S3, CloudWatch)	AWS Fundamentals & Cloud Concepts	AWS global infrastructure; Regions, Availability Zones, Edge Locations; Shared Responsibility Model; IAM roles in engineering organizations; Well-Architected Framework basics (Security, Reliability, Cost, Performance, Operational Excellence)
	Identity & Access Management (IAM)	IAM users, groups, roles & policies; JSON policy documents, allow/deny evaluation logic; Service-linked roles; Least-privilege access design; IAM Access Analyzer; Credential lifecycle management; MFA and best practices; Temporary credentials (STS, AssumeRole)
	Networking with Amazon VPC	VPC, subnets (public/private), route tables; Internet Gateway (IGW) vs NAT Gateway; Security groups vs NACLs; Elastic IPs; DNS & Route 53 basics; VPC flow logs; High-level architecture patterns: 3-tier VPC Private app tier with public API Database subnet isolation
	Storage with Amazon S3	Bucket creation & naming rules; Public access block, policies, ACLs; Object lifecycle management; Versioning, replication & cross-region features; Encryption (SSE-S3, SSE-KMS); Presigned URLs; S3 as static web hosting; Storage classes & cost optimization (S3 Standard, IA, Glacier); Event notifications (S3 → Lambda/SNS/SQS)
	Resource Monitoring & Observability with CloudWatch	CloudWatch metrics (CPU, memory, network, custom metrics); Dashboards & alarms; CloudWatch Logs, log groups, log insights (querying logs); Retention policies Distributed tracing basics; Integration with Lambda, ECS, EC2, RDS, API Gateway; EventBridge for operational automations
	Security, Compliance & Governance	KMS encryption (keys, policies, rotation); Secrets Manager vs Parameter Store; GuardDuty, Config and Security Hub basics; Tagging strategies (cost centers, ownership, environment); Cost Explorer & budgets Golden Rules: Never store secrets in repos Lock down security groups Enforce least privilege in IAM
	Deployment & Operational Patterns	Packaging and deploying cloud workloads; Automated provisioning (IaC: CloudFormation overview); Serverless event-driven architectures; Logging and monitoring patterns for production services; Troubleshooting: networking issues, IAM Deny errors, S3 access errors
AWS Lambda & Serverless	Serverless Fundamentals	What “serverless” means (no server mgmt, automatic scaling, cost-per-use); Event-driven architecture concepts; When to choose serverless vs container vs traditional compute; Key AWS serverless services

		overview: Lambda, API Gateway, EventBridge, SQS, SNS, DynamoDB (contextual), S3 triggers
	AWS Lambda Foundations	Lambda execution model (runtime, handler, context object); Supported languages & Java runtime fundamentals; Memory, timeout, concurrency configuration; Ephemeral filesystem (/tmp); Environment variables & configuration management; Cold starts — what they are and how they occur
	Writing & Packaging Lambda Functions	Maven/Gradle build for Lambda; Using AWS SDK for Java; Structuring request/response payloads; Logging (JSON logs, correlation IDs); Packaging options: ZIP deployment Container image deployment Layers (shared libraries, dependencies)
	Integrating Lambda with Event Services	API Gateway → Lambda (REST/HTTP APIs); S3 → Lambda (object created/updated events); SQS → Lambda (polling & batch processing); SNS → Lambda (pub/sub); EventBridge → Lambda (scheduled events, rule-based routing); Stream processing: Kinesis / DynamoDB Streams (conceptual)
	Security, Permissions & Access Control	Lambda execution role (IAM) ; Resource policies for Lambda (invocation permissions); VPC-enabled Lambdas (ENI creation, subnet selection, security groups); KMS for parameter encryption; Least privilege practices & common anti-patterns
	Performance, Reliability & Operations	Reducing cold starts (memory tuning, provisioned concurrency); Error handling models: Retries Dead-letter queues (DLQ) Event replay Timeouts & throttling behavior; Scaling characteristics (parallel executions, burst limits); Observability (CloudWatch Logs, metrics, structured logging); X-Ray traces for distributed tracing
	Deployment & Automation	Infrastructure as Code basics (SAM & Serverless Framework); SAM templates (Resources, Functions, Events); Packaging & deployment workflows; Versioning & aliases (blue/green deployments); Automated integration with CI/CD pipelines; Rollbacks & safe deployment strategies
	Use Cases & Architecture Patterns	CRUD APIs using Lambda + API Gateway; File processing pipelines (S3 ingest → Lambda → Aurora/S3/SNS); Event-driven microservices workflows; Asynchronous messaging (SNS/SQS → Lambda); Scheduled tasks and cron-based automation; Lambda orchestrations with Step Functions (high-level)

Module/Week 6 (Day 26 to 30) – CI/CD with Jenkins, Containers, Kubernetes & Project

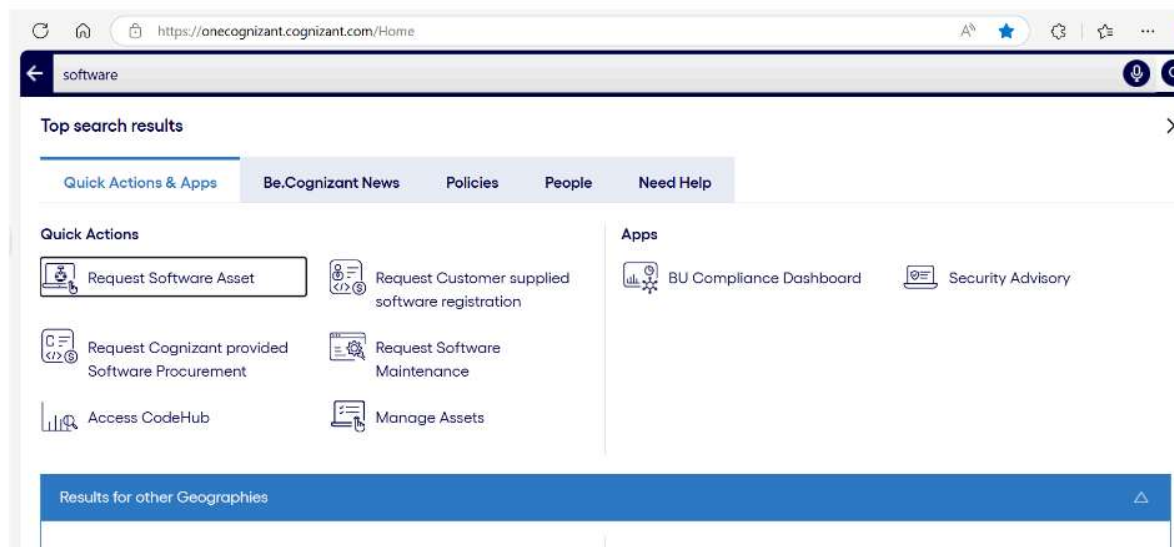
Courses/Modules	Topics	Sub-Topics
CI/CD with Jenkins	CI/CD Fundamentals	What is Continuous Integration, Continuous Delivery & Deployment; Benefits of automation in software engineering; Common CI/CD pipeline stages (build, test, quality gate, package, deploy); Artifact management concepts (versioning, immutability); Role of CI/CD in Agile/DevOps environments
	Jenkins Architecture & Components	Jenkins master/controller vs agents; Agent types (static, dynamic, ephemeral); Jenkins plugins ecosystem overview; Build executors, workspaces & workspace cleanup; Pipelines vs Freestyle jobs — when to choose what; Managing Jenkins via UI vs Jenkinsfile
	Creating Declarative Jenkins Pipelines	Structure of a Jenkinsfile (pipeline, agent, stages, steps); Environment variables & credentials; Parameters and input controls; Shared libraries (reusable pipeline logic); Using tools: Maven, Gradle, NodeJS, Docker; Workspace management & archiving artifacts
	Integrating Code Quality & Testing	Unit test execution (JUnit reports); Code coverage (JaCoCo/Cobertura); Static code analysis (Checkstyle, PMD, SpotBugs); SonarQube integration (quality gates & thresholds) Failing builds based on quality rules; Post-build actions & reporting
	SCM & Triggering Pipelines	Git integration via HTTPS/SSH; Webhooks (GitHub/GitLab/Bitbucket → Jenkins); Poll SCM vs push triggers; Branch-based workflows (feature branches, release branches); Multi-branch pipelines (PR builds, merge validation)
	CI/CD for Containerized Workloads	Building Docker images inside Jenkins; Pushing images to ECR/DockerHub; Scanning container images for vulnerabilities; Using Docker agent vs Docker inside Docker; Building artifacts for Kubernetes deployment
	Deployments & Promotion Workflows	Deploying artifacts to test/stage/prod; Using environment-specific variables; Blue-green deployment basics; Canary deployments (high-level); Rollback strategies; Promotion using approval gates
	Security, Governance & Maintenance	Jenkins credential store & secrets handling; Role-based access control (RBAC); Least-privilege permissions for jobs and agents; Hardening Jenkins (disabling CLI, enabling HTTPS, plugin audits); Backup & restore strategies; Scaling Jenkins with distributed agents
	Monitoring & Observability	Pipeline logs & build logs; Build trends, test reports, failure analytics; Integrating with CloudWatch, ELK, or other log systems; Notifications: Email, Slack, Teams; Troubleshooting failed builds (logs, console output, retries)
Containers & Kubernetes	Containerization Fundamentals	What are containers & why they matter; Containers vs Virtual Machines; Docker architecture & lifecycle;

		Writing production-ready Dockerfiles (multi-stage builds, caching); Image tagging, versioning & security scanning; OCI image registry basics (Docker Hub, ECR, GHCR); Running, inspecting & troubleshooting containers
	Kubernetes Architecture & Core Concepts	Kubernetes control plane components (API Server, etcd, Scheduler, Controller Manager); Worker node components (kubelet, kube-proxy, container runtime); Kubernetes object model understanding; Desired state vs actual state; Labels, annotations & selectors; Deploying workloads on managed Kubernetes (EKS/GKE/AKS — high level)
	Workload Resources (Pods, Deployments, ReplicaSets)	Pod lifecycle & restart policies; Multi-container pod patterns (sidecar, ambassador, adapter); ReplicaSets & scaling behavior; Deployments & rollout strategies; Pausing/resuming rollouts; Rollbacks, version history
	Networking in Kubernetes	Cluster networking model; CNI basics; Service types: ClusterIP, NodePort, LoadBalancer; DNS resolution inside clusters; Ingress & ingress controllers (Nginx, ALB); Network policies (isolation, traffic control)
	Configuration & Secrets Management	ConfigMaps (environment variables, mounted files); Secrets (Opaque, TLS, Docker registry credentials); Security considerations (base64 is not encryption); Managing environment-specific configuration; External Secrets (conceptual – AWS Secrets Manager / HashiCorp Vault integration patterns)
	Storage & Persistence	Volumes vs Persistent Volumes (PVs); Persistent Volume Claims (PVCs); StorageClasses & dynamic provisioning; Using cloud provider storage (EBS, EFS, Azure Disk/File, GCP Persistent Disk); Data retention & stateful workloads (high-level)
	Observability & Debugging	Logs: kubectl logs, multi-container log inspection; Events & describing resources; Probes: readiness, liveness, startup; Metrics server & HPA (Horizontal Pod Autoscaler); K8s Dashboard (conceptual); Using tracing tools (Cloud provider tools or OpenTelemetry — high-level)
	Security, Governance & Best Practices	Role-Based Access Control (RBAC); Namespaces for environment separation; Service Accounts & Pod Security Standards; Network segmentation via network policies; Image scanning, admission controllers (conceptual); Resource requests/limits (CPU/memory); Pod security best practices (runAsUser, root user avoidance)
	Deployment Patterns & Automation	Manifest files (YAML); Package management with Helm (charts, values, releases); GitOps fundamentals (ArgoCD/Flux — conceptual); Blue-green, rolling & canary deployments; Automated CI/CD to Kubernetes via Jenkins or GitHub Actions; Zero-downtime deployments & rollback management

In your final week of technical training, you will be expected to complete a project which will showcase your learning throughout the Cognizant technical training programme. You will need to evidence each of the Activities recorded under each Topic. A full project brief will be shared with you separately.

Software installation Guidelines

Firstly go to onecognizant.com, and search for 'Request Software Asset' (see screen shot below)



Most fields should be auto populated when you select the arrow down and you need to complete each field entering in dates and project codes.

The screenshot shows the Cognizant IT Self Service form. At the top, there is a navigation bar with links: Home, Requests, Knowledge, My Favorites, and My Tickets. The form has several fields: 'Required Till Date' (12-31-2026 00:00:00) with a calendar icon and a tooltip 'Show Calendar for Required Till Date', 'Code' (1000220728), 'Client Name' (Cognizant Academy), and 'Vertical Name' (Internal). Below these is the 'Request Details' section with radio buttons for 'Individual' (selected) and 'Bulk Request'. There is also a checkbox for 'IS VIRTUAL SERVER ? If you are requesting for a VIRTUAL SERVER, please enable check box and select the server in VM Name'. The bottom of the form shows a browser address bar with the URL: https://ctscprod.service-now.com/ss?id=ss_home_page_servicenow.

Then you need to find the software that you want to install, please note that the search function is not great and you may need to search for the owner of the software in order to find it i.e. 'Microsoft Visual Studio' rather than 'Visual Studio'



Under Business justification please put something along the lines of 'I am a UKI graduate studying on the Cognizant Programme. I need this software to complete my technical training over the next 6 weeks.'

Please make sure you choose a version of software that has no cost associated to it, as any with costs will likely not get authorized.

For this particular track the following is recommended software to request to be installed

Skill / Module	Software / Tools Required
Core Java	JDK 18+, IntelliJ IDEA Community Edition, Maven/Gradle, Git CLI, Git GUI (GitHub Desktop/Sourcetree), Lombok (optional), SpotBugs/Checkstyle plugins
JUnit & Test Automation	JUnit 5, Mockito, JaCoCo, SonarLint, IntelliJ Test Runner, Maven/Gradle (test dependencies)
REST API Using JSON	Spring Boot, Postman/Insomnia/Bruno, JSON Viewer/Formatter, Swagger/OpenAPI UI, MapStruct (optional), Maven/Gradle
PostgreSQL & Aurora DB	PostgreSQL Server, pgAdmin 4, DBeaver, Flyway/Liquibase, psql CLI, AWS RDS Console (for Aurora), SQL Shell
JavaScript Essentials	VS Code, Node.js LTS, NPM, Live Server Extension, ESLint, Prettier, Jest (testing), Browser DevTools
AWS Core (IAM, VPC, S3, CloudWatch)	AWS Console Access, AWS CLI, AWS IAM, VPC Dashboard, S3 Console, CloudWatch Logs & Metrics, AWS CloudFormation (optional), AWS Educate/AWS Classroom
AWS Lambda & Serverless	AWS Lambda Console, API Gateway Console, S3 Event Triggers, EventBridge, AWS SAM CLI, Serverless Framework (optional), AWS

	Toolkit (IntelliJ/VS Code), CloudWatch Logs & X-Ray
Jenkins (CI/CD)	Jenkins LTS, Jenkins Pipeline Plugin, Git Plugin, Blue Ocean (optional), JUnit Plugin, JaCoCo Plugin, Credentials Binding Plugin, Docker, SonarQube (optional), Nexus/Artifactory (optional)
Kubernetes (Containers & Orchestration)	Docker Desktop/Minikube, kubectl CLI, Helm, K9s, Lens, Container Registry (Docker Hub or AWS ECR), YAML Lint (optional)

Then select submit and you will either be shared a link which you can use to download the software directly or someone from the GSD team will find time to connect with you, run admin access on your laptop and install the necessary software.

Often the software is available for self installation and you can check on this through 'my to do's' when you open OneCognizant and click on the menu at the top left corner of the screen.

If you run into any issues getting software installed please reach out to ITAMROW (Cognizant) ITAMROW@cognizant.com ; IT UK (Cognizant) ITUK@cognizant.com for support.